MIS41110 Project Guide: Trading Desk and Agent

Oct 29, 2026 · @Zina

1. Before you start

The project is complete and tested: a desk program, an agent and the report pieces. It passes 10 of 10 checks once your group code is in. Do these five things first.

1. **Keep all nine data files together.** `VOLT_news.txt` and `ZNTH_news.txt` are now in the project's `data/training/` folder, so every market runs in all three regimes. Copy all nine files whenever you move the project to another laptop; `selfcheck.py` skips its data runs if your report market's news file is missing.
2. **Put your real group code in `agent.py`.** Line 21 says `GROUP_CODE = "GXX-XXXX"`. Replace it with the code from your group-lock email, exactly as spelled. Never type the four characters after the dash into a terminal or a filename.
3. **Fill in the two name comments** under it: `# Group:` with every member's full name, and `# Reference:` with the member who will make live edits to `agent.py` at the interview.
4. **Declare the AI help in your report.** The rubric requires an AI declaration. State that the code was written with Claude and then reviewed, run and changed by the group. Each member must be able to explain the parts they claim; sections 3, 4 and 7 of this guide are written for that.
5. **Use Python 3.11 or newer** with numpy 2 and pandas 3, like the competition. Install with `python3 -m pip install numpy pandas matplotlib` (Windows: `python -m pip ...`).

The folder layout

Every `.py` file sits in one folder, with no subfolders of code. The data goes in `data/training/` beside it.

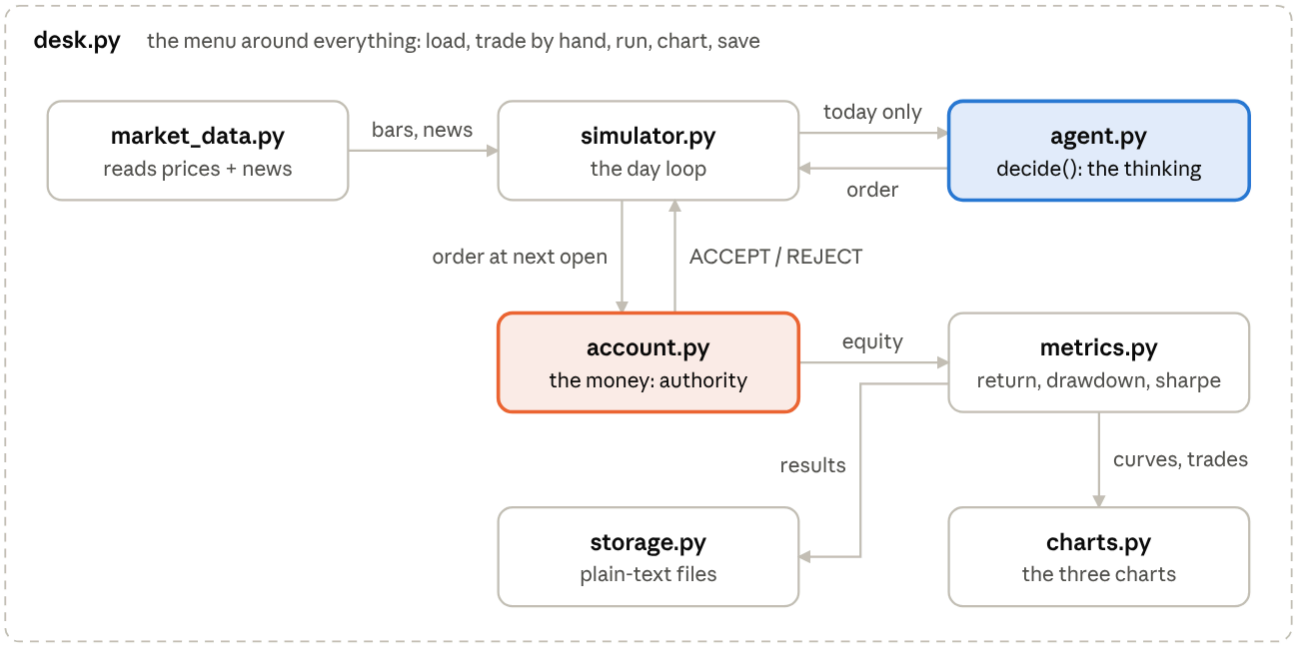
```
desk_v2/
    agent.py            the competition entry (submit)
    desk.py             START HERE: the text menu (submit)
    market_data.py      loading prices and news (submit)
    account.py          cash, shares, fee, ACCEPT/REJECT (submit)
    simulator.py        the back-test loop (submit)
    metrics.py          returns, moving average, drawdown, sharpe (submit)
    charts.py           the three charts (submit)
    storage.py          save/load in plain text (submit)
    benchmarks.py       BaseAgent, HoldAgent, BuyAndHoldAgent (submit)
    check_project.py    14 plain-assert checks (submit)
    agent_unittest.py   course tool (keep beside, do NOT submit)
    selfcheck.py        course tool (keep beside, do NOT submit)
    data/training/      the nine data files (do NOT submit)
```

The program also creates `session.txt`, `results.txt`, `leaderboard.txt` and the chart pictures (`price_NOVA.png` and so on) in the same folder when you use it. It runs in the terminal only: there is no window program.

2. The big picture

The program splits into a *thinking* side (the agent) and an *authority* side (the account and the desk). The agent only proposes orders; only the `Account` ever changes cash or shares. That split is rubric item 2, and it is how the competition works too: the competition plays the authority and calls your `decide()`.

The agent only proposes orders; only the Account changes money



how the files fit together · one back-test day

Each day the simulator hands the agent only today; the agent's order goes to the Account, which fills it at the next open and replies ACCEPT or REJECT.

File	Class / main functions	Its one job	Rubric items
`desk.py`	`Desk`, `ask`, `ask_number`, `main`	The text menu; validates what the user types	7, 8
`agent.py`	`TradingAgent.decide`	Decides BUY / SELL / HOLD each day	4
`account.py`	`Account.execute`, `check_order`	Cash, shares, the 0.1% fee, ACCEPT / REJECT	2, 3
`simulator.py`	`run_backtest`, `RunResult`, `Trade`	The day loop with no look-ahead	3, 11
`market_data.py`	`load_market`, `Market`, `find_price_file`	Reads the CSV and news files	5
`metrics.py`	`daily_returns`, `moving_average`, `max_drawdown`, `sharpe_like`	The numbers, built by hand	9

File	Class / main functions	Its one job	Rubric items
`charts.py`	`price_with_trades`, `equity_curve`, `drawdown`	The three matplotlib charts	9
`storage.py`	`save_session`, `load_session`, `append_result`, `save_leaderboard`	Plain-text files that survive a restart	6
`benchmarks.py`	`BaseAgent`, `HoldAgent`, `BuyAndHoldAgent`	Agents to compare against (inheritance)	2
`check_project.py`	11 `test_` functions	Assert tests of the arithmetic	7

Imports only go one way. `desk.py` imports everything else; `simulator.py` imports `account` and `metrics`; `charts.py` imports `metrics`. Nothing imports `desk.py`, and `agent.py` imports nothing at all. That one-way flow is why each file can be tested and explained on its own.

Why separate files at all? Each file answers one question: "where is the money changed?" (account), "where are files read?" (market_data, storage), "where are numbers computed?" (metrics). In the interview, that lets each member own a file and answer for it.

3. agent.py, explained

The agent trades on the news when it can read it, and on dips below a 10-day moving average when it can only see prices. Three safety rules keep it from ever falling to half its best value. It never fell more than 27% on any training run.

Why this strategy (the evidence)

We tested ideas on the training data before writing the agent. Use these findings in the report's design section.

- **The news works.** Today's news score correlates about +0.2 with tomorrow's move (NOVA 0.23, QBIT 0.19). Days with a positive score rose 0.35% to 0.54% on average the next day; negative days fell by about the same.
- **Trend-following lost money.** A 10/30 moving-average crossover lost on all four markets (sharpe-like -0.006 to -0.075). Every 20-day momentum figure was negative, so these markets tend to bounce back after a fall.
- **So, in price-only runs, buy dips.** Buying when the close is 3% under its 10-day average and selling once it is back above was the most robust rule we tried.

- **Buy-and-hold trips the drawdown gate.** It fell 75% on VOLT and 87% on QBIT: worst-rank in the competition. Hence the safety rules.

The settings block

Every number the strategy uses is a named constant at the top of the file. A live edit at the interview is then usually a one-line change.

Constant	Value	Meaning
`FEE`	0.001	0.1% charged on every buy and sell
`FRACTION_IN_SHARES`	0.70	At most 70% of our money goes into shares
`MA_WINDOW`	10	The moving average uses the last 10 closes
`DIP_NEEDED`	0.03	A "dip" is a close 3% or more below that average
`SAFETY_MARGIN`	1.02	Size buys as if the price were 2% higher, because we fill tomorrow
`STOP_LEVEL`	0.75	Sell everything if our money falls below 75% of its best
`COOL_OFF_DAYS`	10	After that stop, wait 10 days before buying again
`NO_BUY_LEVEL`	0.60	Below 60% of our best value, stop buying altogether
`WORD_SCORES`	dict	The 28 non-neutral words of `sentiment_words.csv`: +1 good, -1 bad

The word list is typed into the file because `agent.py` may not read files. Neutral words score 0, so they are left out.

What `__init__` remembers

The agent only ever sees today, so anything it needs later lives on `self`:

- `self.closes`: every closing price so far (its own history, for the moving average).
- `self.last_price`: the most recent price it knows.
- `self.previous_cash`, `self.previous_shares`: yesterday's balances, used to work out a fill price in the news-only run.
- `self.best_value`: the highest value its money has reached.
- `self.cool_off_left`: days still to wait after a stop.

`decide()` in four steps

```
def decide(self, bar, headlines, cash, shares, last_response):
    # 1. remember the price
    self.learn_price_from_fill(cash, shares)
    if bar is not None:
        self.closes.append(bar["close"])
        self.last_price = bar["close"]
    # 2. the signal: +1 be in, -1 be out, 0 no view
    if headlines is not None:
        signal = self.news_signal(headlines)
    elif bar is not None:
        signal = self.price_signal()
    else:
        return ("HOLD", 0)
    # 3. safety: the stop, then the cool-off
    ...
    # 4. act: sell all on a bad signal, buy on a good one
    if signal < 0 and shares > 0:
        return ("SELL", -1)
    if signal > 0 and shares == 0:
        return self.buy_order(cash)
    return ("HOLD", 0)
```

1. **Remember.** If there is a bar, the close is stored. In the news-only run there is no bar, so `learn_price_from_fill` works out the price from yesterday's fill (below).
2. **Signal.** When there are headlines (news-only and price+news runs), the news decides. With prices only, the moving-average dip rule decides. With neither, the agent holds. This is how it copes with all three regimes and never touches a `None`.
3. **Safety.** The value of our money is `cash + shares * last_price`. If it is a new best, `best_value` is raised. If we hold shares and the value is under 75% of the best, the agent sells everything and sets a 10-day cool-off. While the cool-off lasts, it holds.
4. **Act.** Only two state changes exist: out to in (buy) and in to out (sell everything). It never adds to a position, which keeps fees low and the logic easy to trace.

The two signals

`news_signal(headlines)` is the week-4 review-scoring pattern. It lower-cases each headline, replaces `:` and `,` with spaces, splits it into words and adds up `WORD_SCORES.get(word, 0)`. A positive total gives +1, negative gives -1, zero gives 0. The colon matters: in "report token: QBIT set to bearish" the word `token:` must not swallow the colon.

`price_signal()` asks `moving_average(10)` for the mean of the last 10 closes, built with a plain loop (no pandas `.rolling`). It then computes `distance = close / average - 1`. A distance of -0.03 or lower gives +1 (a dip: buy). A distance above 0 gives -1 (recovered: sell). Anything in between gives 0. Before day 10 there is no average, so it gives 0.

Sizing a buy: `buy_order(cash)`

```
health = (cash / self.best_value - NO_BUY_LEVEL) / (1 - NO_BUY_LEVEL)   # clipped to 0..1
money_to_use = FRACTION_IN_SHARES * health * cash
cost_per_share = self.last_price * (1 + FEE) * SAFETY_MARGIN
count = int(money_to_use / cost_per_share)
```

- `health` is 1.0 at our best value and falls to 0.0 at 60% of it. After losses the agent buys smaller, then stops buying. This is what keeps it away from the 50% drawdown gate.
- `int()` rounds down to whole shares. The fee and the 2% margin make sure the order is affordable at tomorrow's open, so it is rarely rejected.
- With no price known yet (the news-only run's first buy), it buys a single share to learn the price.

Learning the price without a bar: `learn_price_from_fill`

In the news-only run the agent never sees a price, but it can see what a trade cost. Worked example: yesterday 0 shares and 9,500.00 cash; today 5 shares and 8,999.50 cash. The fill cost 500.50 for 5 shares including the fee, so price = 500.50 / (5 × 1.001) = 100.00. After a sale it uses the cash gained and 0.999 instead.

Results on the training markets

These are our own back-test figures on all four markets and all three regimes, matching `selfcheck.py` to the cent. None of the 12 runs fell more than 26.5%; buy-and-hold fell 74.8% on VOLT and 86.5% on QBIT.

Market	Regime	Final equity	Sharpe-like	Max drawdown	Buy-and-hold final
VOLT (volatile)	price only	17,273.18	+0.053	18.7%	3,444.76
VOLT	news only	15,418.12	+0.048	20.7%	3,444.76
VOLT	price + news	16,698.12	+0.055	21.0%	3,444.76
ZNTH (bull)	price only	10,860.69	+0.020	10.0%	11,756.01
ZNTH	news only	11,326.70	+0.024	13.1%	11,756.01
ZNTH	price + news	11,880.71	+0.031	12.5%	11,756.01
NOVA (sideways)	price only	11,140.34	+0.051	3.3%	10,571.16

Market	Regime	Final equity	Sharpe-like	Max drawdown	Buy-and-hold final
NOVA	news only	15,989.29	+0.114	6.0%	10,571.16
NOVA	price + news	16,051.92	+0.115	5.8%	10,571.16
QBIT (bear)	price only	7,471.05	-0.061	26.5%	1,921.74
QBIT	news only	12,929.84	+0.043	23.7%	1,921.74
QBIT	price + news	13,050.44	+0.044	23.1%	1,921.74

On 200 made-up crash markets the worst drawdown was 33.8%. Each run takes about 0.003 s, far under the competition's 0.67 s per run. No part of your grade depends on these numbers; they only matter for the informal leaderboard.

4. The desk program, file by file

The desk is nine small files, each with one job. Read them in the order below: each one only uses the ones before it.

For every function explained one step at a time, see Every function explained.

account.py: the authority

`Account` holds `__cash` and `__shares`. The **two** leading underscores make them private (week 7): from outside the class, `acct.__cash` doesn't reach them. Outside code reads them with `get_cash()` and `get_shares()`, and there is no `set_cash`. That is encapsulation, exactly like the cafe's `Register`.

- `execute(order, price)` first calls `check_order`, which refuses anything that is not `(action, amount)` with action BUY / SELL / HOLD and a real number. It **replies** `"REJECT bad order: ..."` instead of crashing (week 5's "raise, or reply?").
- `__buy` (private): -1 means "as many as I can afford": `int(cash / (price * 1.001))`, rounded down. Example from the contract: $500 at $25 buys 19 shares, not 20 (20 would cost 500.50). If the cost is over the cash, it returns `"REJECT currency balance: "` plus the cash rounded to the cent, and changes nothing.
- `__sell` (private): -1 sells everything; selling more than you hold returns `"REJECT stock owned: "` plus the shares held. A sale pays `amount * price * 0.999`.
- The arithmetic is written in the same order as the official checker (`amount * price * (1 + FEE)`), so our desk's figures match `selfcheck.py` to the cent (NOVA: 16,051.92 in both).

market_data.py: reading the files

- `load_market(ticker)` finds the folder, then calls `load_prices` and `load_news`, and returns a `Market` object.
- `load_prices` uses `pd.read_csv` (taught). It checks that all six columns exist, converts each column to a numpy array *before* indexing (the Series-slicing trap), and builds one dict per day in exactly the shape `decide()` receives. A missing or negative price is replaced by the day before's close, with a warning.
- `load_news` is plain week-2/3 file reading. It skips the header, splits each line once with `split(",", 1)` so a comma inside a headline can never break it, and groups headlines by day in a dict. A bad line is skipped with a warning.
- A missing news file is not fatal: the market loads with no headlines and a note. A missing price file gives back `None` and a message that says what to fix (e.g. "no price file NOVA.csv in data/training/ ..."), so the desk prints it and carries on.
- It never edits or converts the data files, so a held-back market loads unchanged (rubric item 5).

simulator.py: the back-test loop

`run_backtest(agent, market, cash, shares, use_prices, use_news)` is the heart of rubric item 3:

```
for t in range(len(market.bars) - 1):          # every day but the last
    bar = market.bars[t] if use_prices else None
    headlines = market.headlines_on(t) if use_news else None
    order = agent.decide(bar, headlines, account.cash, account.shares,
last_response)
    fill_price = market.bars[t + 1]["open"]     # no look-ahead
    last_response = account.execute(order, fill_price)
    result.equity.append(account.equity(market.bars[t + 1]["close"]))
```

- The agent sees only day `t`; its order fills at the *next* day's open. That is "no look-ahead".
- The loop skips the last day, because its order would fill on a day that does not exist.
- Setting `use_prices` or `use_news` to False gives the price-only and news-only regimes.
- A crash inside the agent or a malformed order is caught, recorded in `result.error`, and stops the run with a clear message instead of crashing the desk.
- `RunResult` keeps the equity curve, the list of `Trade` objects, the reject count and the highest share of money ever in shares. Its `__str__` prints the results block.

metrics.py: the numbers, by hand

Function	How it works	Example
`daily_returns(values)`	`np.array` first, then `today = v[1:]`, `yesterday = v[:-1]`, `today / yesterday - 1`	[100, 110, 99] gives [0.10, -0.10]
`moving_average(values, w)`	A running total: add the new day, subtract the day leaving the window	[1, 2, 3, 4, 5], w=3 gives [2, 3, 4]
`total_return(equity)`	last / first - 1	10,000 to 12,500 gives 0.25
`drawdown_series(equity)`	Track the best value so far; each day's (value - best) / best	
`max_drawdown(equity)`	The lowest value of that series (negative)	[100, 120, 90, 110] gives -0.25
`sharpe_like(equity)`	Mean of the equity's daily returns / their numpy standard deviation; 0 if they never move	

None of them prints or reads files, so each can be tested alone. No `pct_change`, `shift`, `rolling` or `cumprod` appears anywhere.

charts.py: the three charts

- `price_with_trades`: the close in gray, the 10-day moving average in orange, green triangles up for buys and red triangles down for sells, with a legend.
- `equity_curve`: cash + shares × close per day, with a dashed line at the starting value.
- `drawdown`: the drawdown series, shaded, with the maximum in the title.
- Each has a title and labelled axes, is saved as a PNG in the folder you run from (e.g. `price_NOVA.png`), and is shown with `plt.show()`, which waits until you close the window.

storage.py: plain-text persistence

File	Format	Written how
`session.txt`	`key=value` lines: ticker, day, cash, shares, start_cash, start_shares	Overwritten (`"w"`) on each save

File	Format	Written how
`results.txt`	`name,final_equity,trades`, one line per saved run	Appended (`"a"`)
`leaderboard.txt`	`rank,name,final_equity`, best first	Rebuilt from results.txt each save

The leaderboard uses the supplement's recipe: build `[final, name]` pairs, `sort()`, then `reverse()`. A missing file means "nothing saved yet"; a damaged session gives back `None` and the desk says so; a damaged results line is skipped.

benchmarks.py: inheritance

`BaseAgent` is a base class with a `name`, a `__str__`, and a `decide` that raises `NotImplementedError` (abstract, week 7). `HoldAgent` and `BuyAndHoldAgent` inherit from it, call `BaseAgent.__init__(self, ...)` by name, and override `decide`. The desk can run either to compare with ours. They are kept out of `agent.py`, which must stand alone.

desk.py: the menu

- `Desk` holds the session: the loaded `Market`, the current day, the manual `Account`, the manual trades, the last agent run and the latest event.
- One method per menu option; `run()` maps the choice to the method in a dict and prints the status line after every action.
- **Manual trades fill at today's close** (options 4 and 5). **Agent runs fill at the next open** and use a fresh account from the starting balances, so a run never changes your manual session.
- `ask_number` loops until it gets a sensible number: it refuses text, NaN, infinity, negatives (except -1) and absurdly large amounts, each with a message. Fractional share counts are refused too.
- `ask` treats the end of input (Ctrl-D) as quit, and `main` catches Ctrl-C, so the desk always ends with "Goodbye."
- A final `except Exception` around each action reports any unexpected error and keeps the desk open.
- `python3 desk.py` starts with 10,000 cash and 0 shares. `python3 desk.py 5000 10` sets other balances; bad arguments fall back to the defaults with a message.

5. Run, test and demo it

Five commands cover everything, run from the project folder. On Windows type `python` where these say `python3`.

```
python3 check_project.py    14 assert checks; ends "All 14 checks passed."
python3 agent_unittest.py   protocol check; 10/10 once GROUP_CODE is set
python3 selfcheck.py        + three real runs on your report market
python3 desk.py             the desk itself
python3 desk.py 5000 10     the desk with other starting balances
```

What we saw in testing. With a test group code, `selfcheck.py` gave "All checks passed" and "deterministic" on NOVA (11,140.34 / 15,989.29 / 16,051.92) and QBIT (7,471.05 / 12,929.84 / 13,050.44). Your figures may differ by a cent on another numpy/pandas version; that is normal. Paste the three "ran end-to-end" lines from your own run into the report.

The week-12 demonstration, step by step

This is the demo guide's sequence, using our menu:

1. `python3 desk.py` with no arguments. It prints "Welcome. Starting cash 10000.0, shares 0.0." and the menu.
2. **1** then `NOVA`: "Loaded NOVA (750 days)." plus today's headlines and their mood.
3. **4** then `10`: buys 10 shares at today's close and prints `-> ACCEPT`. The status line shows cash 8,985.69 and equity 9,998.99 (the 1.01 fee).
4. **4** then `999` to show a reject: `-> REJECT currency balance: 8985.69`, and nothing changes.
5. **3** a few times to step through days; the headlines print each day.
6. **6**, Enter, Enter: runs TradingAgent with price + news and prints final equity, total return, sharpe-like, max drawdown and trades.
7. **7**: the price chart with buy and sell triangles. Close the window to continue.
8. **8**: the equity curve, then the drawdown chart.
9. **9** then `s`: saves `session.txt`, `results.txt` and `leaderboard.txt`. Then **0** to quit.
10. Start `python3 desk.py` again, **9** then `l`: the session comes back on the same day with the same balances, and the leaderboard prints.

To show error handling, try a ticker that does not exist, the letters `abc` as an amount, `-5` shares, or menu choice `77`. Each gets a clear message and the desk carries on.

Before you submit

- [ ] Real `GROUP_CODE` plus the `# Group:` and `# Reference:` names in `agent.py`

- [ ] `VOLT_news.txt` and `ZNTH_news.txt` in `data/training/`
- [ ] `python3 selfcheck.py` says "All checks passed"; its three lines are in the report
- [ ] Optional: one upload to the online check, to confirm `agent.py` loads on its own
- [ ] Upload the `.py` files one by one (no zip, no folder), plus `Gxx_report.pdf`
- [ ] Do not upload the data, `agent_unittest.py`, `selfcheck.py` or the generated `.txt` files
- [ ] Deadline: Friday 20 November, 23:59

6. The rubric and the report

Every one of the 12 code items has a place in the code. The report is the part you still have to write; drafts for its trickier pieces follow.

#	Rubric item (points)	Where it is met
1	Structure and decomposition (12)	Ten single-purpose files; short functions; numpy/pandas only where they help
2	OOP and separation (12)	Agent thinks, `Account` decides; private `__cash` with `get_cash()`; `__str__` on `Market`, `Account`, `RunResult`, `Trade`, `BaseAgent`; inheritance and abstraction in `benchmarks.py`
3	Back-test correctness (12)	`simulator.py`: next-open fills, 0.1% fee, exact reply strings; manual trades at today's close
4	Agent conformance (6)	`agent.py` imports nothing, reads nothing, passes all 10 checks
5	Data loading (8)	`market_data.py`: header and types checked, missing news survived, data never edited
6	File I/O and persistence (7)	`storage.py`: session, results and leaderboard in plain text
7	Error handling (8)	`try` / `except` for `FileNotFoundError` and `ValueError`; replies instead of crashes (`load_market`, `check_order`); `ask_number`; `assert` in `starting_balances`, the account and the checks
8	User interface (9)	The text menu with every brief section 1 action
9	Analytics and plots (6)	`metrics.py` by hand; three charts in `charts.py`

#	Rubric item (points)	Where it is met
10	Style and comments (7)	Plain names, a header comment per file, docstrings
11	Complexity (6)	The note below
12	Report, UML, manual (7)	The checklist below; `report/activity_diagram.png`

Report checklist

- [ ] Cover page: title, every member's name and student ID
- [ ] Contribution statement: who wrote, tested and owns each file
- [ ] User manual, 1 to 3 pages (adapt section 5 of this guide)
- [ ] UML activity diagram: use `report/activity_diagram.png`
- [ ] Complexity note naming the optimization (draft below)
- [ ] Library justification (draft below)
- [ ] AI declaration (draft below)
- [ ] Self-check record: the three "ran end-to-end" lines from your `selfcheck.py` run
- [ ] File name: `G` + your two digits + `_report.pdf`, nothing after the dash

Draft: complexity note

> The back-test loop in `simulator.run_backtest` visits each of the n days once and does a fixed amount of work per day, so a run is O(n). The agent's per-day work is constant except its 10-day moving average, which is O(10), so a full run stays O(n). Our optimization is in `metrics.moving_average`: instead of re-adding the whole window every day, which is O(n × w), it keeps a running total, adding the new day and subtracting the day that leaves the window, which is O(n). We avoid nested loops over the history, and we never rebuild a pandas DataFrame inside the loop. A 750-day run takes about 0.003 seconds.

Draft: library justification

> We used only libraries installed for the module. **pandas** reads the price CSV with its header (`read_csv`), because that is what it is for. **numpy** holds the closes as arrays so returns can be computed by position with slicing, and gives the standard deviation for the sharpe-like score. **matplotlib** draws the three charts. **math** and **numbers** (standard library) check that order amounts are real, finite numbers. We did not use pandas' `pct_change`, `shift`, `rolling` or `cumprod`: we built those calculations ourselves, as the brief requires. `agent.py` imports nothing, so it runs on its own in the competition.

Draft: AI declaration (edit to match what you really did)

> We used Claude (Anthropic) to help write the code and this documentation. The group reviewed every file, ran the tests and the self-checks, and is responsible for the submission. [Describe what you changed yourselves, e.g. "we adjusted the settings after testing" or "we rewrote the menu text".] Each member can explain the files they claim in the contribution statement.

Contribution statement template

Member	Owns (answers questions on)	Also did
Name 1 (reference for agent.py)	agent.py	Strategy testing
Name 2	account.py, simulator.py	Back-test checks
Name 3	market_data.py, storage.py	Error handling
Name 4	metrics.py, charts.py	Report figures
Name 5	desk.py, benchmarks.py	User manual, demo

7. Interview preparation

The code walkthrough and live edit are 45% of the interview mark, and each member answers for the files they claim. Practise the hand traces and edits below on paper and on the laptop until they take under two minutes.

Two hand traces to rehearse

Trace 1: a price-only buy. Cash 10,000, 0 shares, best value 10,000. The last 10 closes average 100.00 and today's close is 96.50.

1. `distance = 96.50 / 100.00 - 1 = -0.035`, which is ⩽ -0.03, so `price_signal()` returns +1.
2. Value = 10,000, not under 75% of the best, so no stop. No cool-off.
3. Signal +1 and 0 shares, so `buy_order(10000)`.
4. `health = (10000/10000 - 0.6) / 0.4 = 1.0`; `money_to_use = 0.7 × 1.0 × 10000 = 7000`.
5. `cost_per_share = 96.50 × 1.001 × 1.02 = 98.528`; `int(7000 / 98.528) = 71`.
6. Returns `("BUY", 71)`. It fills tomorrow at the open.

Trace 2: a news day. Headlines: "NOVA surge as market rally" and "NOVA fear in late token update". Scores: surge +1, rally +1, fear -1, the rest 0. Total +1, so the signal is +1. If we already hold shares, it returns `("HOLD", 0)`; if not, it buys.

Likely questions, with short answers

Question	Answer
What does the agent trade on?	News word counts when it can read news; dips below the 10-day moving average when it only sees prices.
Why is the word list inside agent.py?	agent.py may not read files; the competition loads it alone.
Why does it never put in 100%?	The competition scores a run worst-rank if equity ever falls to half its best. 70% plus the stop keeps it far away from that.
Why buy dips rather than follow the trend?	On our data a crossover lost on all four markets, and momentum was negative everywhere, so prices tended to bounce back.
Why does the desk fill at the close but the agent at the next open?	A person at the desk trades at today's price; the agent decides after the close, so the first price it could get is tomorrow's open. Anything else is look-ahead.

Question	Answer
Why is `__cash` private?	Only the Account may change money; the agent and the desk ask, the Account decides. Two underscores hide it, `get_cash()` lets others read it, and there is no `set_cash` (week 7's Register).
Why `split(",", 1)`?	It splits only at the first comma, so a comma inside a headline cannot break the line.
Why `np.array` before slicing?	Slicing a pandas Series aligns by index label and silently gives zeros.
Big-O of the back-test?	O(n) in days: one pass, constant work per day.
Big-O of `moving_average`?	O(n) with the running total; O(n × w) if the whole window were re-added each day.
How do you know it is deterministic?	It uses no randomness, no clock and no state between runs; selfcheck.py runs it twice and compares.
What happens if decide() crashes?	In our desk, `run_backtest` catches it and reports the day and the error. In the competition the run scores worst-rank.
Where is inheritance used?	`HoldAgent` and `BuyAndHoldAgent` extend `BaseAgent`, calling `BaseAgent.__init__(self, name)` and overriding `decide`, which the base leaves abstract with `raise NotImplementedError`.

Live edits to practise

The reference member makes the edit; the group talks it through. Run `python3 agent_unittest.py` after each one.

1. **"Hold whenever cash is under 100."** In `decide()`, right after step 1, add:

```
if cash < 100:
    return ("HOLD", 0)
```

2. **"Use a 20-day moving average."** Change `MA_WINDOW = 10` to `MA_WINDOW = 20`.
3. **"Never put more than half the money in shares."** Change `FRACTION_IN_SHARES = 0.70` to `0.50`.

4. **"Make the agent sell when the news is bad."** It already does: step 4's `if signal < 0 and shares > 0: return ("SELL", -1)`. Show the line, then trace it with a headline containing "crash".
5. **"Handle a missing price."** In `decide()`, change `if bar is not None:` to `if bar is not None and bar.get("close") is not None:`. On the desk side, point to `load_prices`, which already fills a missing price with the day before's close.
6. **"Count 'record' as a bad word."** In `WORD_SCORES`, change `"record": 1` to `"record": -1`.

A last check before the interview

Bring one laptop with Python, numpy, pandas and matplotlib, holding exactly the files you submitted, the training data in `data/training/`, and `agent_unittest.py` and `selfcheck.py` beside them. Run the whole demo once on that laptop the day before.